Задание 2
Часть 1. Создание изолированного процесса средствами ядра Linux 3
Часть 2. Создание и запуск OCI bundle средствами runc 8
Часть 3. Образы контейнеров 17
Часть 4. Дополнительное задание 24
Дополнительная информация 30

Задание

Работа разбита на этапы:

1. Создание изолированного процесса средствами ядра Linux (namespaces, cgroups).

2. Создание и запуск OCI bundle средствами runc.

3. Сборка и анализ образа контейнера.

4. Исследование безопасности контейнеров.

Часть 1. Создание изолированного процесса средствами ядра Linux

Шаг 1.

Используя флаги `--pid`, `--uts`, `--mount`, `--net`, `--ipc` создаются изолированные области видимости ресурсов.

```shell

      sudo unshare --pid --uts --mount --net --ipc --fork /bin/sh

      ps aux

      hostname

```

```
sapman@temp-ubu:/tmp/container$ tar -xzf alpine-minirootfs-3.20.3-x86_64.tar.gz -C rootfs
sapman@temp-ubu:/tmp/container$
sapman@temp-ubu:/tmp/container$
sapman@temp-ubu:/tmp/container$ ls -la
total 3424
drwxrwxr-x  3 sapman sapman    4096 Apr 27 21:40 .
drwxrwxrwt 13 root   root      4096 Apr 27 21:40 ..
-rw-rw-r--  1 sapman sapman 3490290 Sep  6  2024 alpine-minirootfs-3.20.3-x86_64.tar.gz
drwxr-xr-x 19 sapman sapman    4096 Sep  6  2024 rootfs
sapman@temp-ubu:/tmp/container$
sapman@temp-ubu:/tmp/container$ sudo unshare --pid --uts --mount --net --ipc --fork /bin/sh
# mount -t proc proc /proc
#
# ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.0   2808  1920 pts/2    S    21:40   0:00 /bin/sh
root           3  0.0  0.0  11328  4428 pts/2    R+   21:40   0:00 ps aux
#
# hostname
temp-ubu
#
#
```

Шаг 2.

Используя команду `pivot_root` перемещается вся текущая корневая файловая система в отдельную директорию и устанавливается новая файловая система (`Alpine rootfs`) в качестве корня.

```
sapman@temp-ubu:/tmp/container$ sudo unshare --pid --uts --mount --net --ipc --fork /bin/sh
# mount -t proc proc /proc
#
# ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.0   2808  1920 pts/2    S    21:40   0:00 /bin/sh
root           3  0.0  0.0  11328  4428 pts/2    R+   21:40   0:00 ps aux
#
# hostname
temp-ubu
#
# mount --bind /tmp/container/rootfs /tmp/container/rootfs
#
# cd /tmp/container/rootfs
#
# mkdir -p oldroot
#
# pivot_root . oldroot
you have mail
you have mail
you have mail
#
# cd /
#
# ls -la /
total 80
drwxr-xr-x   20 1000     1000          4096 Apr 27 21:42 .
drwxr-xr-x   20 1000     1000          4096 Apr 27 21:42 ..
drwxr-xr-x    2 1000     1000          4096 Sep  6  2024 bin
drwxr-xr-x    2 1000     1000          4096 Sep  6  2024 dev
drwxr-xr-x   17 1000     1000          4096 Sep  6  2024 etc
drwxr-xr-x    2 1000     1000          4096 Sep  6  2024 home
drwxr-xr-x    6 1000     1000          4096 Sep  6  2024 lib
drwxr-xr-x    5 1000     1000          4096 Sep  6  2024 media
drwxr-xr-x    2 1000     1000          4096 Sep  6  2024 mnt
drwxr-xr-x   23 root     root          4096 Apr 27 21:39 oldroot
drwxr-xr-x    2 1000     1000          4096 Sep  6  2024 opt
dr-xr-xr-x    2 1000     1000          4096 Sep  6  2024 proc
drwx------    2 1000     1000          4096 Sep  6  2024 root
drwxr-xr-x    2 1000     1000          4096 Sep  6  2024 run
drwxr-xr-x    2 1000     1000          4096 Sep  6  2024 sbin
drwxr-xr-x    2 1000     1000          4096 Sep  6  2024 srv
drwxr-xr-x    2 1000     1000          4096 Sep  6  2024 sys
drwxrwxr-x    2 1000     1000          4096 Sep  6  2024 tmp
drwxr-xr-x    7 1000     1000          4096 Sep  6  2024 usr
drwxr-xr-x   12 1000     1000          4096 Sep  6  2024 var
#
#
# ps aux
PID   USER     TIME  COMMAND
#
# hostname
temp-ubu
#
# mount -t proc proc /proc
#
# ls /
bin      dev      etc      home     lib      media    mnt      oldroot  opt      proc     root     run      sbin     srv      sys      tmp      usr      var
#
# ps aux
PID   USER     TIME  COMMAND
    1 root      0:00 /bin/sh
   13 root      0:00 ps aux
#
#
```

Это обеспечивает более высокий уровень безопасности, так как после последующего размонтирования (`umount`) старого корня у изолированного процесса физически исчезает доступ к файлам хоста.

Шаг 3.

Ядро идентифицирует каждое пространство имен уникальным номером `inode`. Информация `ls -l /proc/<PID>/ns/`.

```
[sapman@temp-ubu:~$ ps aux | grep "unshare"
root        1614  0.0  0.0  17148  7004 pts/0    S+   21:40   0:00 sudo unshare --pid --uts --mount --net --ipc --fork /bin/sh
root        1615  0.0  0.0  17148  2540 pts/2    Ss   21:40   0:00 sudo unshare --pid --uts --mount --net --ipc --fork /bin/sh
root        1616  0.0  0.0   6128  1980 pts/2    S    21:40   0:00 unshare --pid --uts --mount --net --ipc --fork /bin/sh
sapman      1669  0.0  0.0   7084  2168 pts/1    S+   21:45   0:00 grep --color=auto unshare
[sapman@temp-ubu:~$
[sapman@temp-ubu:~$ ps -ef --forest | grep -A 1 "unshare"
root        1614    1499  0 21:40 pts/0    00:00:00  |           \_ sudo unshare --pid --uts --mount --net --ipc --fork /bin/sh
root        1615    1614  0 21:40 pts/2    00:00:00  |               \_ sudo unshare --pid --uts --mount --net --ipc --fork /bin/sh
root        1616    1615  0 21:40 pts/2    00:00:00  |                   \_ unshare --pid --uts --mount --net --ipc --fork /bin/sh
root        1617    1616  0 21:40 pts/2    00:00:00  |                       \_ /bin/sh
--
sapman      1671    1590  0 21:46 pts/1    00:00:00              \_ grep --color=auto -A 1 unshare
sapman      1354       1  0 21:39 ?        00:00:00 /usr/lib/systemd/systemd --user
[sapman@temp-ubu:~$
[sapman@temp-ubu:~$ ls -l /proc/1617/ns/
ls: cannot open directory '/proc/1617/ns/': Permission denied
[sapman@temp-ubu:~$ sudo ls -l /proc/1617/ns/
total 0
lrwxrwxrwx 1 root root 0 Apr 27 21:48 cgroup -> 'cgroup:[4026531835]'
lrwxrwxrwx 1 root root 0 Apr 27 21:48 ipc -> 'ipc:[4026532210]'
lrwxrwxrwx 1 root root 0 Apr 27 21:48 mnt -> 'mnt:[4026532208]'
lrwxrwxrwx 1 root root 0 Apr 27 21:48 net -> 'net:[4026532212]'
lrwxrwxrwx 1 root root 0 Apr 27 21:48 pid -> 'pid:[4026532211]'
lrwxrwxrwx 1 root root 0 Apr 27 21:48 pid_for_children -> 'pid:[4026532211]'
lrwxrwxrwx 1 root root 0 Apr 27 21:48 time -> 'time:[4026531834]'
lrwxrwxrwx 1 root root 0 Apr 27 21:48 time_for_children -> 'time:[4026531834]'
lrwxrwxrwx 1 root root 0 Apr 27 21:48 user -> 'user:[4026531837]'
lrwxrwxrwx 1 root root 0 Apr 27 21:48 uts -> 'uts:[4026532209]'
[sapman@temp-ubu:~$
[sapman@temp-ubu:~$ ls -l /proc/$$/ns/
total 0
lrwxrwxrwx 1 sapman sapman 0 Apr 27 21:49 cgroup -> 'cgroup:[4026531835]'
lrwxrwxrwx 1 sapman sapman 0 Apr 27 21:49 ipc -> 'ipc:[4026531839]'
lrwxrwxrwx 1 sapman sapman 0 Apr 27 21:49 mnt -> 'mnt:[4026531832]'
lrwxrwxrwx 1 sapman sapman 0 Apr 27 21:49 net -> 'net:[4026531833]'
lrwxrwxrwx 1 sapman sapman 0 Apr 27 21:49 pid -> 'pid:[4026531836]'
lrwxrwxrwx 1 sapman sapman 0 Apr 27 21:49 pid_for_children -> 'pid:[4026531836]'
lrwxrwxrwx 1 sapman sapman 0 Apr 27 21:49 time -> 'time:[4026531834]'
lrwxrwxrwx 1 sapman sapman 0 Apr 27 21:49 time_for_children -> 'time:[4026531834]'
lrwxrwxrwx 1 sapman sapman 0 Apr 27 21:49 user -> 'user:[4026531837]'
lrwxrwxrwx 1 sapman sapman 0 Apr 27 21:49 uts -> 'uts:[4026531838]'
[sapman@temp-ubu:~$
sapman@temp-ubu:~$
```

Если номера `inode` в выводе `ls -l` для хостового процесса и изолированного процесса различаются, это является доказательством того, что процессы находятся в разных областях видимости ресурсов.

Шаг 4.

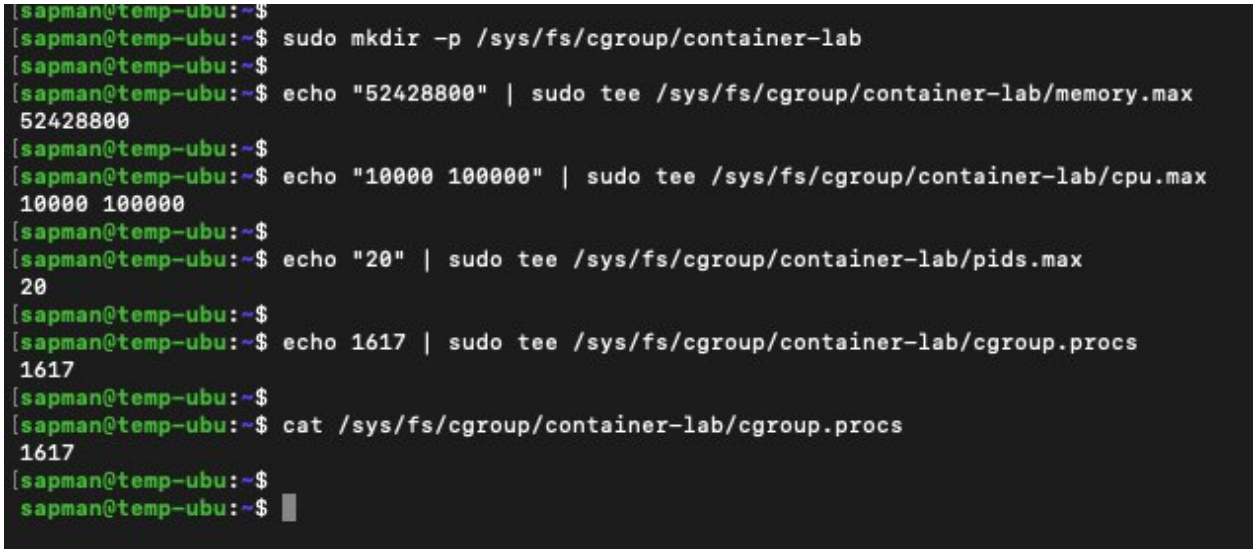
В отличие от `namespaces` (которые ограничивают видимость), контрольные группы (`cgroups`) ограничивают потребление ресурсов.

Ограничители:

- `memory.max`: устанавливает «жесткий» лимит памяти
- `cpu.max`: задает квоту процессорного времени на определенный период (например, 10% от ядра)
- `pids.max`: ограничивает максимальное число процессов для защиты от fork-потребилетей

```shell
```

```
# Создает директорию для новой группы
sudo mkdir -p /sys/fs/cgroup/container-lab

# Устанавливает лимит памяти 50 МБ
echo "52428800" | sudo tee /sys/fs/cgroup/container-lab/memory.max

# Устанавливает лимит CPU (10% от одного ядра)
echo "10000 100000" | sudo tee /sys/fs/cgroup/container-lab/cpu.max

# Ограничивает количество процессов до 20
echo "20" | sudo tee /sys/fs/cgroup/container-lab/pids.max
```



Запись `PID` в файл `cgroup.procs` мгновенно распространяет эти правила на процесс и всех его fork’ов.

```shell
# Помещает изолированный процесс в созданную группу
echo 1617 | sudo tee /sys/fs/cgroup/container-lab/cgroup.procs
```

```

Написанный скрипт представляет собой пример экспоненциального заполнения оперативной памяти (`heap`).

```shell

STUFF="A" # Инициализация переменной (1 байт)

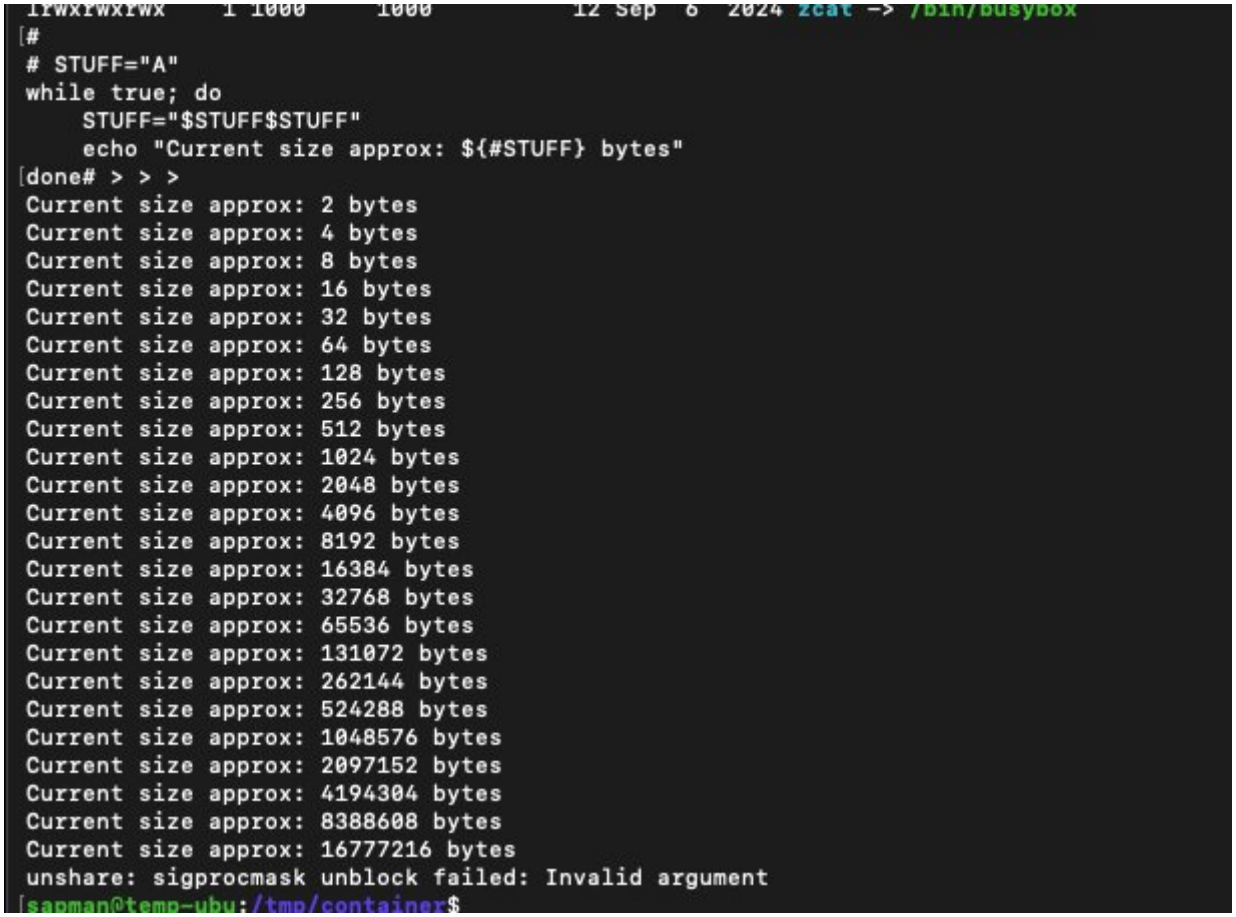
while true; do

STUFF="$STUFF$STUFF" # Конкатенация (удвоение строки в каждой итерации)

echo "Current size approx: ${#STUFF} bytes" # Вывод текущей длины строки

done

```


```

Чтобы защитить остальную систему от нехватки ресурсов, ядро активирует механизм `Out Of Memory (OOM) Killer`, который принудительно завершает самый «прожорливый» процесс в данной группе сигналом `SIGKILL`.

```
[sapman@temp-ubu:~$ cat /sys/fs/cgroup/container-lab/cgroup.procs
1617
[sapman@temp-ubu:~$
[sapman@temp-ubu:~$ cat /sys/fs/cgroup/container-lab/memory.current
8192
[sapman@temp-ubu:~$ cat /sys/fs/cgroup/container-lab/memory.max
52428800
[sapman@temp-ubu:~$
[sapman@temp-ubu:~$ cat /sys/fs/cgroup/container-lab/memory.current
397312
[sapman@temp-ubu:~$
[sapman@temp-ubu:~$ cat /sys/fs/cgroup/container-lab/memory.current
126976
[sapman@temp-ubu:~$
[sapman@temp-ubu:~$ grep "oom" /sys/fs/cgroup/container-lab/memory.events
oom 1
oom_kill 1
oom_group_kill 0
sapman@temp-ubu:~$
```

```shell
grep "oom" /sys/fs/cgroup/container-lab/memory.events
```

Факт принудительной остановки фиксируется в счетчике `oom_kill` файла `memory.events`.

Часть 2. Создание и запуск OCI bundle средствами runc

Шаг 1.

Создание OCI bundle.

```shell
mkdir -p /tmp/oci-bundle/rootfs


# Копирует чистую rootfs Alpine
sudo cp -r /tmp/container/rootfs/* /tmp/oci-bundle/rootfs/
```

```
cd /tmp/oci-bundle


# Генерирует config.json

runc spec
```

```
sapman@temp-ubu:/tmp$
sapman@temp-ubu:/tmp$ sudo cp -r /tmp/container/rootfs/* /tmp/oci-bundle/rootfs/
sapman@temp-ubu:/tmp$
sapman@temp-ubu:/tmp$ cd /tmp/oci-bundle
sapman@temp-ubu:/tmp/oci-bundle$
sapman@temp-ubu:/tmp/oci-bundle$
sapman@temp-ubu:/tmp/oci-bundle$
sapman@temp-ubu:/tmp/oci-bundle$ ls -la
total 12
drwxrwxr-x  3 sapman sapman 4096 Apr 29 10:53 .
drwxrwxrwt 14 root   root   4096 Apr 29 10:53 ..
drwxrwxr-x 19 sapman sapman 4096 Apr 29 10:53 rootfs
sapman@temp-ubu:/tmp/oci-bundle$
sapman@temp-ubu:/tmp/oci-bundle$
sapman@temp-ubu:/tmp/oci-bundle$ ls -la rootfs/
total 76
drwxrwxr-x 19 sapman sapman 4096 Apr 29 10:53 .
drwxrwxr-x  3 sapman sapman 4096 Apr 29 10:53 ..
drwxr-xr-x  2 root   root   4096 Apr 29 10:53 bin
drwxr-xr-x  2 root   root   4096 Apr 29 10:53 dev
drwxr-xr-x 17 root   root   4096 Apr 29 10:53 etc
drwxr-xr-x  2 root   root   4096 Apr 29 10:53 home
drwxr-xr-x  6 root   root   4096 Apr 29 10:53 lib
drwxr-xr-x  5 root   root   4096 Apr 29 10:53 media
drwxr-xr-x  2 root   root   4096 Apr 29 10:53 mnt
drwxr-xr-x  2 root   root   4096 Apr 29 10:53 opt
dr-xr-xr-x  2 root   root   4096 Apr 29 10:53 proc
drwx------  2 root   root   4096 Apr 29 10:53 root
drwxr-xr-x  2 root   root   4096 Apr 29 10:53 run
drwxr-xr-x  2 root   root   4096 Apr 29 10:53 sbin
drwxr-xr-x  2 root   root   4096 Apr 29 10:53 srv
drwxr-xr-x  2 root   root   4096 Apr 29 10:53 sys
drwxr-xr-x  2 root   root   4096 Apr 29 10:53 tmp
drwxr-xr-x  7 root   root   4096 Apr 29 10:53 usr
drwxr-xr-x 12 root   root   4096 Apr 29 10:53 var
sapman@temp-ubu:/tmp/oci-bundle$
sapman@temp-ubu:/tmp/oci-bundle$
sapman@temp-ubu:/tmp/oci-bundle$ runc spec
sapman@temp-ubu:/tmp/oci-bundle$ ls -la
total 16
drwxrwxr-x  3 sapman sapman 4096 Apr 29 10:54 .
drwxrwxrwt 14 root   root   4096 Apr 29 10:53 ..
-rw-rw-r--  1 sapman sapman 2500 Apr 29 10:54 config.json
drwxrwxr-x 19 sapman sapman 4096 Apr 29 10:53 rootfs
sapman@temp-ubu:/tmp/oci-bundle$
sapman@temp-ubu:/tmp/oci-bundle$
sapman@temp-ubu:/tmp/oci-bundle$ cat config.json
{
        "ociVersion": "1.2.1",
```

Шаг 2.

Изучение конфигурации.

```shell

      cat config.json

```

Вывод:

```json

{

    "ociVersion": "1.2.1",

    "process": {

        "terminal": true,

        "user": {

            "uid": 0,

            "gid": 0

        },

        "args": [

            "sh"

        ],

        "env": [

"PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",

            "TERM=xterm"

        ],

        "cwd": "/",

        "capabilities": {

            "bounding": [
```

```
            "CAP_AUDIT_WRITE",
            "CAP_KILL",
            "CAP_NET_BIND_SERVICE"
        ],
        "effective": [
            "CAP_AUDIT_WRITE",
            "CAP_KILL",
            "CAP_NET_BIND_SERVICE"
        ],
        "permitted": [
            "CAP_AUDIT_WRITE",
            "CAP_KILL",
            "CAP_NET_BIND_SERVICE"
        ]
    },
    "rlimits": [
        {
            "type": "RLIMIT_NOFILE",
            "hard": 1024,
            "soft": 1024
        }
    ],
    "noNewPrivileges": true
},
"root": {
    "path": "rootfs",
    "readonly": true
```

```
},
"hostname": "runc",
"mounts": [
    {
        "destination": "/proc",
        "type": "proc",
        "source": "proc"
    },
    {
        "destination": "/dev",
        "type": "tmpfs",
        "source": "tmpfs",
        "options": [
            "nosuid",
            "strictatime",
            "mode=755",
            "size=65536k"
        ]
    },
    {
        "destination": "/dev/pts",
        "type": "devpts",
        "source": "devpts",
        "options": [
            "nosuid",
            "noexec",
            "newinstance",
```

```
            "ptmxmode=0666",
            "mode=0620",
            "gid=5"
        ]
    },
    {
        "destination": "/dev/shm",
        "type": "tmpfs",
        "source": "shm",
        "options": [
            "nosuid",
            "noexec",
            "nodev",
            "mode=1777",
            "size=65536k"
        ]
    },
    {
        "destination": "/dev/mqueue",
        "type": "mqueue",
        "source": "mqueue",
        "options": [
            "nosuid",
            "noexec",
            "nodev"
        ]
    },
```

```
        {
            "destination": "/sys",
            "type": "sysfs",
            "source": "sysfs",
            "options": [
                "nosuid",
                "noexec",
                "nodev",
                "ro"
            ]
        },
        {
            "destination": "/sys/fs/cgroup",
            "type": "cgroup",
            "source": "cgroup",
            "options": [
                "nosuid",
                "noexec",
                "nodev",
                "relatime",
                "ro"
            ]
        }
    ],
    "linux": {
        "resources": {
            "devices": [
```

```
            {
                "allow": false,
                "access": "rwm"
            }
        ]
    },
    "namespaces": [
        {
            "type": "pid"
        },
        {
            "type": "network"
        },
        {
            "type": "ipc"
        },
        {
            "type": "uts"
        },
        {
            "type": "mount"
        },
        {
            "type": "cgroup"
        }
    ],
    "maskedPaths": [
```

```
            "/proc/acpi",
            "/proc/asound",
            "/proc/kcore",
            "/proc/keys",
            "/proc/latency_stats",
            "/proc/timer_list",
            "/proc/timer_stats",
            "/proc/sched_debug",
            "/sys/firmware",
            "/proc/scsi"
        ],
        "readonlyPaths": [
            "/proc/bus",
            "/proc/fs",
            "/proc/irq",
            "/proc/sys",
            "/proc/sysrq-trigger"
        ]
    }
}
```

Шаг 3.

Запуск контейнера через runc.

```shell
      # shell
      cd /tmp/oci-bundle
```

```
sudo runc run lab-container

ps aux

hostname

ls /
```

```
sapman@temp-ubu:~$ cd /tmp/oci-bundle/
sapman@temp-ubu:/tmp/oci-bundle$
sapman@temp-ubu:/tmp/oci-bundle$ sudo runc run lab-container
/ # ps aux
PID   USER     TIME  COMMAND
    1 root      0:00 sh
    7 root      0:00 ps aux
/ #
/ # hostname
runc
/ #
/ # ls /
bin    dev    etc    home   lib    media  mnt    opt    proc   root   run    sbin   srv    sys    tmp    usr    var
/ #
/ #
```

```shell
sudo runc list

sudo runc state lab-container
```

```
sapman@temp-ubu:/tmp/oci-bundle$
sapman@temp-ubu:/tmp/oci-bundle$ sudo runc list
ID              PID         STATUS      BUNDLE            CREATED                          OWNER
lab-container   7875        running     /tmp/oci-bundle   2026-04-29T11:24:51.866013515Z   root
sapman@temp-ubu:/tmp/oci-bundle$
sapman@temp-ubu:/tmp/oci-bundle$
sapman@temp-ubu:/tmp/oci-bundle$ sudo runc state lab-container
{
  "ociVersion": "1.2.1",
  "id": "lab-container",
  "pid": 7875,
  "status": "running",
  "bundle": "/tmp/oci-bundle",
  "rootfs": "/tmp/oci-bundle/rootfs",
  "created": "2026-04-29T11:24:51.866013515Z",
  "owner": ""
}sapman@temp-ubu:/tmp/oci-bundle$
sapman@temp-ubu:/tmp/oci-bundle$
sapman@temp-ubu:/tmp/oci-bundle$
```

Шаг 4.

Сравнение результатов.

Основное различие заключается в методе управления ресурсами ядра.

Часть 1 представляет собой «ручную» сборку контейнера из низкоуровневых примитивов;

Часть 2 использует индустриальный стандарт `OCI (Open Container Initiative)`*.

**OCI — Open Container Initiative, набор открытых стандартов для форматов образов, среды выполнения и распространения контейнеров.*

Характеристика	Часть 1: Примитивы ядра	Часть 2: OCI Bundle и runc
Инструментарий	Утилиты: unshare, mount, pivot_root.	Утилита runc и файл config.json.
Конфигурация	Последовательный ввод команд в терминале.	Декларативное описание в формате JSON.
Изоляция (PID)	Оболочка sh получает случайный PID на хосте.	Внутри контейнера процесс всегда получает PID 1.
Управление ресурсами	Ручная запись PID в файлы /sys/fs/cgroup/..	Автоматическая настройка через секцию resources в конфиге.
Жизненный цикл	Зависит от родительского процесса терминала.	Стандартизирован: creating, running, stopped.

Утилита `runc` является эталонной реализацией спецификации `OCI Runtime`. Она полностью автоматизирует цепочку действий, которую в первой части выполнялось вручную:

- Создание пространств имён (`namespaces`): `runc` самостоятельно вызывает системные вызовы `clone()` или `unshare()` с нужными флагами (`PID`, `NET`, `MNT` и др.), основываясь на списке из `config.json`.
- Подготовка файловой системы: Выполняет системный вызов `pivot_root()` для полной замены корневой ФС на `rootfs`, обеспечивая безопасность и невозможность выхода за пределы контейнера.

- Настройка `cgroups`: Автоматически создает иерархию директорий в `/sys/fs/cgroup`, устанавливает лимиты (`RAM`, `CPU`) и помещает туда процесс контейнера.
- Гранулярная безопасность: Применяет `capabilities`, ограничивая права `root` внутри контейнера.
- Устанавливает `seccomp-профили` для фильтрации опасных системных вызовов.
- Скрывает чувствительные системные пути через `maskedPaths`.
- Запуск процесса: После настройки всех ограничений выполняет `exec()`, превращая текущий изолированный процесс в целевой (например, `/bin/sh`).

Поведение:

В Части 1 созданный процесс (`1617`) был убит ядром (`OOM Killer`) при ручной настройке лимитов.

В Части 2 `runc` позволял мгновенно запустить ту же среду, но с уже готовой структурой: команда `ps aux` внутри контейнера отображает `sh` под `PID 1`, а `hostname` отображает `runc`, согласно конфигурации.

Команда `runc` предоставляет удобные средства мониторинга (`runc list` и `runc state`), которые заменяют ручной поиск `PID` и проверку `inode` в `/proc`.

Часть 3. Образы контейнеров

Шаг 1.

Сборка образа и анализ tar-архива.

```shell
mkdir -p ~/lab-docker && cd ~/lab-docker
```

```Plaintext

# Создает простой Dockerfile используя на основу alpine:3.20

echo -e "FROM alpine:3.20\nRUN echo "First custom layer" > /step1.txt\nRUN echo "Second custom layer" > /step2.txt" > Dockerfile

```

```
[sapman@temp-ubu:~/lab-docker$
[sapman@temp-ubu:~/lab-docker$ ls -la
 total 12
 drwxrwxr-x 2 sapman docker 4096 Apr 30 18:40 .
 drwxr-x--- 6 sapman sapman 4096 Apr 30 18:40 ..
 -rw-rw-r-- 1 sapman docker   90 Apr 30 18:40 Dockerfile
[sapman@temp-ubu:~/lab-docker$
[sapman@temp-ubu:~/lab-docker$
[sapman@temp-ubu:~/lab-docker$ docker build -t lab-image .
 [+] Building 1.9s (7/7) FINISHED
  => [internal] load build definition from Dockerfile
  => => transferring dockerfile: 127B
  => [internal] load metadata for docker.io/library/alpine:3.20
  => [internal] load .dockerignore
  => => transferring context: 2B
  => CACHED [1/3] FROM docker.io/library/alpine:3.20@sha256:d9e853e87e55526f6b2917df91a2115c36dd7c696a35be12163d44e6e2a4b6bc
  => => resolve docker.io/library/alpine:3.20@sha256:d9e853e87e55526f6b2917df91a2115c36dd7c696a35be12163d44e6e2a4b6bc
  => [2/3] RUN echo "First layer" > /step1.txt
  => [3/3] RUN echo "Second layer" > /step2.txt
  => exporting to image
  => => exporting layers
  => => exporting manifest sha256:084bbf37be0b25d933ff6a7f47a2185d67c44cc3ea9be8f5b925842d4a16ee94
  => => exporting config sha256:8f412571eb50a1db4b9bd61bd0bbd3dad1bccd8e3e3d4641ed320ca44ff0bb85
  => => exporting attestation manifest sha256:a51460553a408d2787af6dc07bc6b5f370ddce7c09d6637756660514bfe2e18e
  => => exporting manifest list sha256:e93912f8daea85775bf6f5c7403a1e2c9d631932324de472f1819d699d823159
  => => naming to docker.io/library/lab-image:latest
  => => unpacking to docker.io/library/lab-image:latest
[sapman@temp-ubu:~/lab-docker$
```

```shell

docker build -t lab-image .

docker history lab-image:latest

docker save lab-image:latest -o image.tar

# Распаковка

mkdir unpacked-image && tar -xf image.tar -C unpacked-image

ls -la unpacked-image

```

```
sapman@temp-ubu:~/lab-docker$ docker history lab-image:latest
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
e93912f8daea   29 seconds ago   RUN /bin/sh -c echo "Second layer" > /step2.…   8.19kB    buildkit.dockerfile.v0
<missing>      29 seconds ago   RUN /bin/sh -c echo "First layer" > /step1.t…   8.19kB    buildkit.dockerfile.v0
<missing>      13 days ago      CMD ["/bin/sh"]                                 0B        buildkit.dockerfile.v0
<missing>      13 days ago      ADD alpine-minirootfs-3.20.10-x86_64.tar.gz …   8.47MB    buildkit.dockerfile.v0
sapman@temp-ubu:~/lab-docker$
sapman@temp-ubu:~/lab-docker$ docker save lab-image:latest -o image.tar
sapman@temp-ubu:~/lab-docker$
sapman@temp-ubu:~/lab-docker$ mkdir unpacked-image && tar -xf image.tar -C unpacked-image
sapman@temp-ubu:~/lab-docker$
sapman@temp-ubu:~/lab-docker$ ls -la unpacked-image/
total 24
drwxrwxr-x 3 sapman sapman 4096 Apr 30 18:50 .
drwxrwxr-x 3 sapman docker 4096 Apr 30 18:50 ..
drwxr-xr-x 3 sapman sapman 4096 Jan  1  1970 blobs
-rw-r--r-- 1 sapman sapman  362 Jan  1  1970 index.json
-rw-r--r-- 1 sapman sapman  375 Jan  1  1970 manifest.json
-r--r--r-- 1 sapman sapman   30 Jan  1  1970 oci-layout
sapman@temp-ubu:~/lab-docker$
```

```
sapman@temp-ubu:~/lab-docker$
sapman@temp-ubu:~/lab-docker$ ls -la
total 3588
drwxrwxr-x 3 sapman docker    4096 Apr 30 19:10 .
drwxr-x--- 6 sapman sapman    4096 Apr 30 19:10 ..
-rw-rw-r-- 1 sapman docker      90 Apr 30 18:40 Dockerfile
-rw------- 1 sapman sapman 3648000 Apr 30 18:49 image.tar
-rw-rw-r-- 1 sapman sapman    6592 Apr 30 19:10 inspect_out.json
drwxrwxr-x 3 sapman sapman    4096 Apr 30 18:50 unpacked-image
sapman@temp-ubu:~/lab-docker$
sapman@temp-ubu:~/lab-docker$
sapman@temp-ubu:~/lab-docker$ tree
Command 'tree' not found, but can be installed with:
sudo snap install tree  # version 2.1.3+pkg-5852, or
sudo apt  install tree  # version 2.1.1-2ubuntu3.24.04.2
See 'snap info tree' for additional versions.
sapman@temp-ubu:~/lab-docker$
sapman@temp-ubu:~/lab-docker$ ls -la
total 3588
drwxrwxr-x 3 sapman docker    4096 Apr 30 19:10 .
drwxr-x--- 6 sapman sapman    4096 Apr 30 19:10 ..
-rw-rw-r-- 1 sapman docker      90 Apr 30 18:40 Dockerfile
-rw------- 1 sapman sapman 3648000 Apr 30 18:49 image.tar
-rw-rw-r-- 1 sapman sapman    6592 Apr 30 19:10 inspect_out.json
drwxrwxr-x 3 sapman sapman    4096 Apr 30 18:50 unpacked-image
sapman@temp-ubu:~/lab-docker$
sapman@temp-ubu:~/lab-docker$ ls -la unpacked-image/
total 24
drwxrwxr-x 3 sapman sapman 4096 Apr 30 18:50 .
drwxrwxr-x 3 sapman docker 4096 Apr 30 19:10 ..
drwxr-xr-x 3 sapman sapman 4096 Jan  1  1970 blobs
-rw-r--r-- 1 sapman sapman  362 Jan  1  1970 index.json
-rw-r--r-- 1 sapman sapman  375 Jan  1  1970 manifest.json
-r--r--r-- 1 sapman sapman   30 Jan  1  1970 oci-layout
sapman@temp-ubu:~/lab-docker$ ls -la unpacked-image/blobs/
total 12
drwxr-xr-x 3 sapman sapman 4096 Jan  1  1970 .
drwxrwxr-x 3 sapman sapman 4096 Apr 30 18:50 ..
drwxr-xr-x 2 sapman sapman 4096 Jan  1  1970 sha256
sapman@temp-ubu:~/lab-docker$ ls -la unpacked-image/blobs/sha256/
total 3588
drwxr-xr-x 2 sapman sapman    4096 Jan  1  1970 .
drwxr-xr-x 3 sapman sapman    4096 Jan  1  1970 ..
-r--r--r-- 1 sapman sapman     857 Jan  1  1970 084bbf37be0b25d933ff6a7f47a2185d67c44cc3ea9be8f5b925842d4a16ee94
-r--r--r-- 1 sapman sapman 3630321 Jan  1  1970 25f1d6b1951ac8eb3740558fe94cb83d377bdadf95fd9f98b50d2e1b96130471
-r--r--r-- 1 sapman sapman     167 Jan  1  1970 3e7cce85f68d470b35a2b533625917f1f528ef89e579587ca2388feb5ec15eab
-r--r--r-- 1 sapman sapman    1080 Jan  1  1970 8f412571eb50a1db4b9bd61bd0bbd3dad1bccd8e3e3d4641ed320ca44ff0bb85
-r--r--r-- 1 sapman sapman     120 Jan  1  1970 9c82d603462c79ca5e17b50558ad3f49ac60393535596973fe5d94265bc27aa9
-r--r--r-- 1 sapman sapman     564 Jan  1  1970 a51460553a408d2787af6dc07bc6b5f370ddce7c09d6637756660514bfe2e18e
-r--r--r-- 1 sapman sapman     119 Jan  1  1970 e2d9321b25d8caf7eaeaee93b535aa1781baca7f29601f018afc2935dd9ee1e4
-r--r--r-- 1 sapman sapman     855 Jan  1  1970 e93912f8daea85775bf6f5c7403a1e2c9d631932324de472f1819d699d823159
-r--r--r-- 1 sapman sapman    1035 Jan  1  1970 eaa5a4db747871cc63696cf54ec1f5b27dc910a73786a5c03f53e648c7869f06
sapman@temp-ubu:~/lab-docker$
sapman@temp-ubu:~/lab-docker$
```

Шаг 2.

Поиск слоёв

```shell
      docker run -itd --name lab-container-overlay lab-image sh
```

Видимо, версия `Docker` и/или `Ubuntu 24.04` развёрнутая в `Azure` имеет особенности в отображением `OverlayFS` и обычная команда `docker inspect` здесь не подходит.

Использовалась команда `mount | grep overlay`

```Plaintext
      $ mount | grep overlay
      overlay on
/var/lib/docker/rootfs/overlayfs/13c07b29b2438936f5a1ee77f8fbdc69cabe756e5933
5d3df3276b7954fb5bc2 type overlay
(rw,relatime,lowerdir=/var/lib/containerd/io.containerd.snapshotter.v1.overla
yfs/snapshots/28/fs:/var/lib/containerd/io.containerd.snapshotter.v1.overlayf
s/snapshots/27/fs:/var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/
snapshots/26/fs:/var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/sn
apshots/6/fs,upperdir=/var/lib/containerd/io.containerd.snapshotter.v1.overla
yfs/snapshots/29/fs,workdir=/var/lib/containerd/io.containerd.snapshotter.v1.
overlayfs/snapshots/29/work,nouserxattr)
```

```
[sapman@temp-ubu:~/lab-docker$
[sapman@temp-ubu:~/lab-docker$ docker ps
CONTAINER ID   IMAGE       COMMAND   CREATED          STATUS          PORTS     NAMES
13c07b29b243   lab-image   "sh"      7 minutes ago    Up 7 minutes              lab-container-overlay
[sapman@temp-ubu:~/lab-docker$
[sapman@temp-ubu:~/lab-docker$
[sapman@temp-ubu:~/lab-docker$ docker inspect lab-container-overlay | grep "Dir"
            "WorkingDir": "/",
[sapman@temp-ubu:~/lab-docker$
[sapman@temp-ubu:~/lab-docker$ sudo docker inspect lab-container-overlay | grep -iE "lower|upper|merged"
[sapman@temp-ubu:~/lab-docker$
[sapman@temp-ubu:~/lab-docker$
[sapman@temp-ubu:~/lab-docker$ docker info | grep "Storage Driver"
 Storage Driver: overlayfs
[sapman@temp-ubu:~/lab-docker$
[sapman@temp-ubu:~/lab-docker$ docker inspect lab-container-overlay | grep "/var/lib/docker/overlay2"
[sapman@temp-ubu:~/lab-docker$
[sapman@temp-ubu:~/lab-docker$ mount | grep overlay
overlay on /var/lib/docker/rootfs/overlayfs/13c07b29b2438936f5a1ee77f8fbdc69cabe756e59335d3df3276b7954fb5bc2 type overlay (rw,relatime,lowerdir=/var/lib/containerd/io.containerd.snap
shotter.v1.overlayfs/snapshots/28/fs:/var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/27/fs:/var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/2
6/fs:/var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/6/fs,upperdir=/var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/29/fs,workdir=/var/lib/co
ntainerd/io.containerd.snapshotter.v1.overlayfs/snapshots/29/work,nouserxattr)
[sapman@temp-ubu:~/lab-docker$
[sapman@temp-ubu:~/lab-docker$ docker inspect lab-container-overlay > inspect_out.json
[sapman@temp-ubu:~/lab-docker$ less inspect_out.json
[sapman@temp-ubu:~/lab-docker$
[sapman@temp-ubu:~/lab-docker$ cat inspect_out.json |grep Dir
            "WorkingDir": "/",
[sapman@temp-ubu:~/lab-docker$ cat inspect_out.json |grep -i 'Dir'
            "WorkingDir": "/",
[sapman@temp-ubu:~/lab-docker$ docker ps
CONTAINER ID   IMAGE       COMMAND   CREATED          STATUS          PORTS     NAMES
13c07b29b243   lab-image   "sh"      22 minutes ago   Up 22 minutes             lab-container-overlay
[sapman@temp-ubu:~/lab-docker$
[sapman@temp-ubu:~/lab-docker$ docker exec lab-container-overlay touch /home/findIt.txt
[sapman@temp-ubu:~/lab-docker$
[sapman@temp-ubu:~/lab-docker$
[sapman@temp-ubu:~/lab-docker$ sudo docker inspect lab-container-overlay | grep -iE "lower|upper|merged"
[sapman@temp-ubu:~/lab-docker$
[sapman@temp-ubu:~/lab-docker$ sudo ls -l /var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/29/fs
total 4
drwxr-xr-x 2 root root 4096 Apr 30 19:17 home
[sapman@temp-ubu:~/lab-docker$
[sapman@temp-ubu:~/lab-docker$ sudo ls -l /var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/29/fs/home
total 0
-rw-r--r-- 1 root root 0 Apr 30 19:17 findIt.txt
[sapman@temp-ubu:~/lab-docker$
 sapman@temp-ubu:~/lab-docker$
```

Файл успешно найден:

```shell

      sudo ls -l /var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/29/fs/home

```

Шаг 3.

Скачивание удалённого секрета.

```shell

      echo "KEEP_MY_SECRET" > secret.txt

      cat Dockerfile

      docker build -t secret-leaker .

      docker history secret-leaker


      # Разные вариации dive
```

```
dive secret-leaker:latest

sudo dive secret-leaker:latest

docker history secret-leaker:latest
```

С `dive` пришлось повозиться, но так корректно запустить её и не удалось. Было принято Докерезивать пользование утилиты.

Вывод команды `dive`:

```Plaintext
$ sudo dive secret-leaker:latest

Image Source: docker://secret-leaker:latest

Fetching image... (this can take a while for large images)

Handler not available locally. Trying to pull 'secret-leaker:latest'...

cannot fetch image

cannot find docker client executable
```

```
[sapman@temp-ubu:~/lab-docker$
[sapman@temp-ubu:~/lab-docker$ echo "KEEP_MY_SECRET" > secret.txt
[sapman@temp-ubu:~/lab-docker$
[sapman@temp-ubu:~/lab-docker$ cat Dockerfile
 FROM alpine
 COPY secret.txt /secret.txt
 RUN rm /secret.txt
[sapman@temp-ubu:~/lab-docker$
 sapman@temp-ubu:~/lab-docker$
```

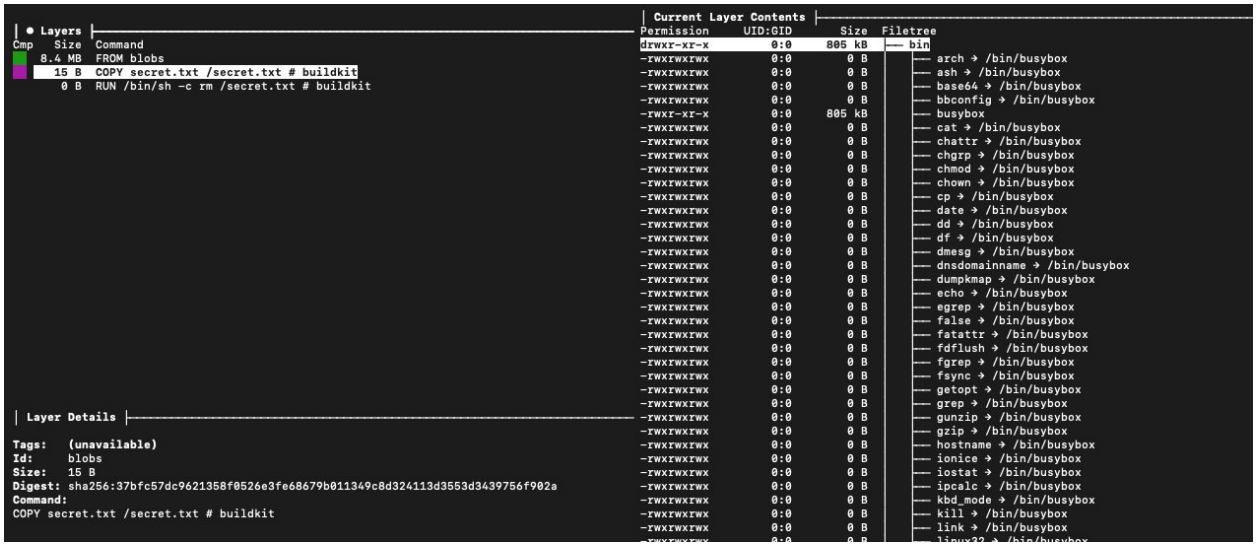
```
sapman@temp-ubu:~/lab-docker$
sapman@temp-ubu:~/lab-docker$ docker build -t secret-leaker .
[+] Building 3.0s (8/8) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 96B
 => [internal] load metadata for docker.io/library/alpine:latest
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [internal] load build context
 => => transferring context: 52B
 => [1/3] FROM docker.io/library/alpine:latest@sha256:5b10f432ef3da1b8d4c7eb6c487f2f5a8f096bc91145e68878dd4a5019afde11
 => => resolve docker.io/library/alpine:latest@sha256:5b10f432ef3da1b8d4c7eb6c487f2f5a8f096bc91145e68878dd4a5019afde11
 => => sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb 3.86MB / 3.86MB
 => => extracting sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb
 => [2/3] COPY secret.txt /secret.txt
 => [3/3] RUN rm /secret.txt
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:9778af036c0491480f28279eddc07bc9188914c5d7c4b7407e9bf566ab856408
 => => exporting config sha256:8c88faa3387977da911fbb8694a97a5f34bc22f858d5d8bf1d2dc43d44ddb74a
 => => exporting attestation manifest sha256:421f4de265e5b55f6b9726f08908876f95e2bf1c3ed179b613809aa908a8dae4
 => => exporting manifest list sha256:47ff69ff7d454c84b8e80aab53fb6493cb72b3b8f70333a15b6c86318dee42ae
 => => naming to docker.io/library/secret-leaker:latest
 => => unpacking to docker.io/library/secret-leaker:latest
sapman@temp-ubu:~/lab-docker$
sapman@temp-ubu:~/lab-docker$
```

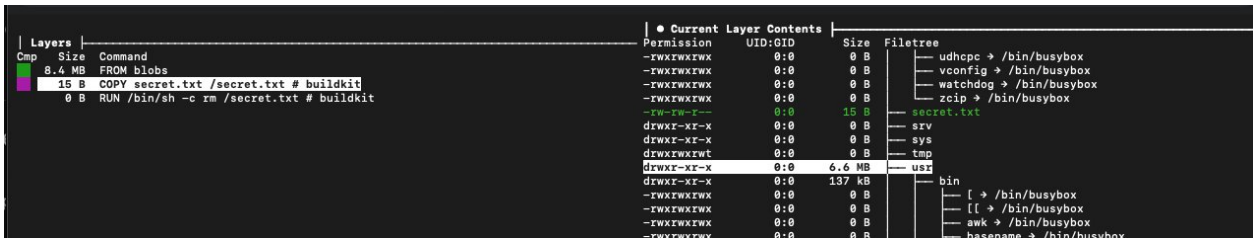
```
sapman@temp-ubu:~/lab-docker$ docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
sapman@temp-ubu:~/lab-docker$
sapman@temp-ubu:~/lab-docker$ docker images

IMAGE                  ID             DISK USAGE   CONTENT SIZE   EXTRA
secret-leaker:latest   55e94b0ae08c         13MB         3.87MB
sapman@temp-ubu:~/lab-docker$
sapman@temp-ubu:~/lab-docker$ dive 55e94b0ae08c
Image Source: docker://55e94b0ae08c
Fetching image... (this can take a while for large images)
Handler not available locally. Trying to pull '55e94b0ae08c'...
cannot fetch image
cannot find docker client executable
sapman@temp-ubu:~/lab-docker$
sapman@temp-ubu:~/lab-docker$ dive secret-leaker:latest
Image Source: docker://secret-leaker:latest
Fetching image... (this can take a while for large images)
Handler not available locally. Trying to pull 'secret-leaker:latest'...
cannot fetch image
cannot find docker client executable
sapman@temp-ubu:~/lab-docker$
sapman@temp-ubu:~/lab-docker$ sudo dive secret-leaker:latest
Image Source: docker://secret-leaker:latest
Fetching image... (this can take a while for large images)
Handler not available locally. Trying to pull 'secret-leaker:latest'...
cannot fetch image
cannot find docker client executable
sapman@temp-ubu:~/lab-docker$
sapman@temp-ubu:~/lab-docker$ docker history secret-leaker:latest
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
55e94b0ae08c   34 minutes ago   RUN /bin/sh -c rm /secret.txt # buildkit        4.1kB     buildkit.dockerfile.v0
<missing>      34 minutes ago   COPY secret.txt /secret.txt # buildkit          8.19kB    buildkit.dockerfile.v0
<missing>      2 weeks ago      CMD ["/bin/sh"]                                 0B        buildkit.dockerfile.v0
<missing>      2 weeks ago      ADD alpine-minirootfs-3.23.4-x86_64.tar.gz /…   9.11MB    buildkit.dockerfile.v0
sapman@temp-ubu:~/lab-docker$
sapman@temp-ubu:~/lab-docker$
```

Запуск `dive` внутри Docker:

```shell
docker run --rm -it \
    -v /var/run/docker.sock:/var/run/docker.sock \
    wagoodman/dive:latest secret-leaker:latest
```

```




Ручная распаковка:

```shell

# Сохраняет в tar

docker save secret-leaker -o leaked.tar



# Распаковывает

mkdir leaked_content && tar -xvf leaked.tar -C leaked_content

cd leaked_content

cat manifest.json | python3 -m json.tool



# Проверка файла

tar -tf
blobs/sha256/e6afcd4d54117de78f4570a35772d57501c7b09682b50a8c273a6424bca
070e1 | grep secret.txt
```
```

```
# Получение и чтение файла

tar -xvf
blobs/sha256/e6afcd4d54117de78f4570a35772d57501c7b09682b50a8c273a6424bca
070e1 secret.txt

cat secret.txt
```

```
sapman@temp-ubu:~/lab-docker$
sapman@temp-ubu:~/lab-docker$ ls -la
total 20
drwxrwxr-x 2 sapman docker 4096 Apr 30 19:53 .
drwxr-x--- 7 sapman sapman 4096 Apr 30 19:52 ..
-rw-rw-r-- 1 sapman sapman   59 Apr 30 19:33 Dockerfile
-rw-rw-r-- 1 sapman docker   90 Apr 30 18:40 Dockerfile_old
-rw-rw-r-- 1 sapman sapman   15 Apr 30 19:34 secret.txt
sapman@temp-ubu:~/lab-docker$
sapman@temp-ubu:~/lab-docker$
sapman@temp-ubu:~/lab-docker$ docker save secret-leaker -o leaked.tar
sapman@temp-ubu:~/lab-docker$ mkdir leaked_content && tar -xvf leaked.tar -C leaked_content
blobs/
blobs/sha256/
blobs/sha256/43fb5c17ec75d8febd5b3674b79f47bd792236fcab2a3c2ef33becf28eec54cc
blobs/sha256/55e94b0ae08c5f52fe11fb599a22926f8a469d3a32c11fdadd7ebf0e7466730c
blobs/sha256/6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb
blobs/sha256/6a69a14a999d661f4ab92dcd1dec3492ea6659a5f12c613e4250c1189b03a549
blobs/sha256/757a297e3ee1191a8e20590044aea0952bbbdf5206314439443ac8ad8d6437f5
blobs/sha256/a1c2fc6539ba7e79e1543486164bd53a8641ee9beadcc78a28c028847c06207b
blobs/sha256/c88157550a6e7dfabf473965ca0b0da246bb4894003c1e7d1c65bab4c5836eb5
blobs/sha256/dedfe26338099fc3f7a773f6d2874202214c4da6bc21120810ee640d27e6cc2c
blobs/sha256/e6afcd4d54117de78f4570a35772d57501c7b09682b50a8c273a6424bca070e1
index.json
manifest.json
oci-layout
sapman@temp-ubu:~/lab-docker$
sapman@temp-ubu:~/lab-docker$ cd leaked_content
sapman@temp-ubu:~/lab-docker/leaked_content$
sapman@temp-ubu:~/lab-docker/leaked_content$ ls -la
total 24
drwxrwxr-x 3 sapman sapman 4096 Apr 30 20:45 .
drwxrwxr-x 3 sapman docker 4096 Apr 30 20:45 ..
drwxr-xr-x 3 sapman sapman 4096 Jan  1  1970 blobs
-rw-r--r-- 1 sapman sapman  366 Jan  1  1970 index.json
-rw-r--r-- 1 sapman sapman  379 Jan  1  1970 manifest.json
-r--r--r-- 1 sapman sapman   30 Jan  1  1970 oci-layout
sapman@temp-ubu:~/lab-docker/leaked_content$
sapman@temp-ubu:~/lab-docker/leaked_content$
sapman@temp-ubu:~/lab-docker/leaked_content$ cat manifest.json
[{"Config":"blobs/sha256/6a69a14a999d661f4ab92dcd1dec3492ea6659a5f12c613e4250c1189b03a549","RepoTags":["secret-leaker:latest"],"Layers":["blobs/sha256/6a0ac1617861a677b045b7ff8854521
3ec31c0ff08763195a70a4a5adda577bb","blobs/sha256/e6afcd4d54117de78f4570a35772d57501c7b09682b50a8c273a6424bca070e1","blobs/sha256/757a297e3ee1191a8e20590044aea0952bbbdf5206314439443ac
8ad8d6437f5"]}]sapman@temp-ubu:~/lab-docker/leaked_content$
sapman@temp-ubu:~/lab-docker/leaked_content$
sapman@temp-ubu:~/lab-docker/leaked_content$ cat manifest.json | python3 -m json.tool
[
    {
        "Config": "blobs/sha256/6a69a14a999d661f4ab92dcd1dec3492ea6659a5f12c613e4250c1189b03a549",
        "RepoTags": [
            "secret-leaker:latest"
        ],
        "Layers": [
            "blobs/sha256/6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb",
            "blobs/sha256/e6afcd4d54117de78f4570a35772d57501c7b09682b50a8c273a6424bca070e1",
            "blobs/sha256/757a297e3ee1191a8e20590044aea0952bbbdf5206314439443ac8ad8d6437f5"
        ]
    }
]
sapman@temp-ubu:~/lab-docker/leaked_content$
sapman@temp-ubu:~/lab-docker/leaked_content$ tar -tf blobs/sha256/e6afcd4d54117de78f4570a35772d57501c7b09682b50a8c273a6424bca070e1 | grep secret.txt
secret.txt
sapman@temp-ubu:~/lab-docker/leaked_content$
sapman@temp-ubu:~/lab-docker/leaked_content$ tar -xvf blobs/sha256/e6afcd4d54117de78f4570a35772d57501c7b09682b50a8c273a6424bca070e1 secret.txt
secret.txt
sapman@temp-ubu:~/lab-docker/leaked_content$
sapman@temp-ubu:~/lab-docker/leaked_content$ cat secret.txt
KEEP_MY_SECRET
sapman@temp-ubu:~/lab-docker/leaked_content$
```

Шаг 4.

Описание поведения:

- `Иммутабельность` слоев: Каждый слой в Docker — это неизменяемый архив.
- `Copy-on-Write` и `Whiteouts`: Когда выполняется `rm /secret.txt`, Docker не «смотрит» в предыдущий слой, чтобы стереть там данные. Вместо этого в новом (верхнем) слое создается специальный файл-

заглушка (`whiteout`), который говорит системе: «`не показывай этот файл в MergedDir`».

- `Безопасность`: Это главная причина, по которой нельзя передавать секреты через `COPY` или `ENV`, даже если не планируется публиковать образ. Секрет останется в истории навсегда.

Часть 4. Дополнительное задание

Раздел 2.5.

Часть 1.

Флаг `--privileged` отключает практически все механизмы защиты Docker: снимает ограничения `cgroups` на устройства (открывая доступ к `/dev/` хоста), отключает `seccomp/AppArmor` и выдает контейнеру все возможные привилегии (`capabilities`) ядра Linux.

Запуск обычного контейнера

```shell
docker run --rm -it alpine sh -c "apk add --no-cache libcap && capsh --print | grep Current"
```

Наблюдается поведение в разделе `Current IAB:` добавлен флаг «`!`» исключающий данное разрешение к контейнере.

```Plaintext
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
```

```
Digest:
sha256:5b10f432ef3da1b8d4c7eb6c487f2f5a8f096bc91145e68878dd4a5019afde11

Status: Downloaded newer image for alpine:latest

(1/5) Installing libcap2 (2.78-r0)

(2/5) Installing libcap-getcap (2.78-r0)

(3/5) Installing libcap-setcap (2.78-r0)

(4/5) Installing libcap-utils (2.78-r0)

(5/5) Installing libcap (2.78-r0)

Executing busybox-1.37.0-r30.trigger

OK: 8364 KiB in 21 packages

Current:
cap_chown,cap_dac_override,cap_fowner,cap_fsetid,cap_kill,cap_setgid,cap
_setuid,cap_setpcap,cap_net_bind_service,cap_net_raw,cap_sys_chroot,cap_
mknod,cap_audit_write,cap_setfcap=ep

Current
IAB: !cap_dac_read_search,!cap_linux_immutable,!cap_net_broadcast,!c
ap_net_admin,!cap_ipc_lock,!cap_ipc_owner,!cap_sys_module,!cap_sys_rawio
,!cap_sys_ptrace,!cap_sys_pacct,!cap_sys_admin,!cap_sys_boot,!cap_sys_ni
ce,!cap_sys_resource,!cap_sys_time,!cap_sys_tty_config,!cap_lease,!cap_a
udit_control,!cap_mac_override,!cap_mac_admin,!cap_syslog,!cap_wake_alar
m,!cap_block_suspend,!cap_audit_read,!cap_perfmon,!cap_bpf,!cap_checkpoi
nt_restore
```

Запуск привилегированного контейнера

```shell
docker run --rm -it alpine sh -c \
```

```
"apk add --no-cache libcap && capsh --print | grep Current"
```

Все ограничения сняты:

```Plaintext
(1/5) Installing libcap2 (2.78-r0)
(2/5) Installing libcap-getcap (2.78-r0)
(3/5) Installing libcap-setcap (2.78-r0)
(4/5) Installing libcap-utils (2.78-r0)
(5/5) Installing libcap (2.78-r0)
Executing busybox-1.37.0-r30.trigger
OK: 8364 KiB in 21 packages
Current: =ep
Current IAB:
```

Получение доступа к ФС:

```shell
# Создает точку монтирования
mkdir /mnt/host_root

# Монтирует /dev/sda1
mount /dev/sda1 /mnt/host_root
```

```
# Получает директорию локального пользователя на хосте

ls -la /mnt/host_root/home/
```

```
[sapman@temp-ubu:~/lab-docker$
[sapman@temp-ubu:~/lab-docker$ docker run --rm -it --privileged alpine sh
[/ #
[/ # fdisk -l
Found valid GPT with protective MBR; using GPT

Disk /dev/sda: 134217728 sectors,     0
Logical sector size: 512
Disk identifier (GUID): e1e5281c-ef16-4267-a10d-859778606da2
Partition table holds up to 128 entries
First usable sector is 34, last usable sector is 134217694

Number  Start (sector)    End (sector)  Size Name
     1         2099200       134217694 62.9G
    14            2048           10239 4096K
    15           10240          227327  106M
    16          227328         2097152  913M
[/ #
[/ # mkdir /mnt/host_root
[/ #
[/ # mount /dev/sda1 /mnt/host_root
[/ #
[/ # ls -la /mnt/host_root/home/
total 12
drwxr-xr-x    3 root     root          4096 Apr 27 20:31 .
drwxr-xr-x   23 root     root          4096 Apr 30 17:43 ..
drwxr-x---    7 1000     1000          4096 Apr 30 20:46 sapman
[/ #
[/ # ls -la /mnt/host_root/home/sapman/
total 60
drwxr-x---    7 1000     1000          4096 Apr 30 20:46 .
drwxr-xr-x    3 root     root          4096 Apr 27 20:31 ..
-rw-------    1 1000     1000          4826 Apr 29 20:43 .bash_history
-rw-r--r--    1 1000     1000           220 Mar 31  2024 .bash_logout
-rw-r--r--    1 1000     1000          3771 Mar 31  2024 .bashrc
drwx------    2 1000     1000          4096 Apr 27 20:41 .cache
drwx------    3 1000     988           4096 Apr 29 20:12 .docker
-rw-------    1 1000     1000            33 Apr 30 19:10 .lesshst
-rw-r--r--    1 1000     1000           807 Mar 31  2024 .profile
-rw-------    1 1000     1000             0 Apr 30 20:46 .python_history
drwx------    2 1000     1000          4096 Apr 27 20:31 .ssh
-rw-r--r--    1 1000     1000             0 Apr 27 20:42 .sudo_as_admin_successful
-rw-------    1 1000     1000          1527 Apr 30 19:33 .viminfo
-rw-rw-r--    1 1000     1000           177 Apr 29 10:52 .wget-hsts
drwxrwxr-x    3 1000     988           4096 Apr 30 20:45 lab-docker
drwx------    3 1000     1000          4096 Apr 30 19:52 snap
[/ #
/ #
```

Полученный доступ позволяет читать/менять любые файлы хоста (например, `/etc/shadow`), внедрять SSH-ключи или устанавливать руткиты.

Меры защиты:

`Отказ от --privileged (Принцип наименьших привилегий)`: Если контейнеру нужна специфика (например, управление сетью), нужно использовать точечную выдачу прав через флаг `--cap-add=NET_ADMIN` вместо выдачи всех прав.

`Использование User Namespaces (UserNS)`: Эта технология позволяет сделать так, чтобы процесс, запущенный от имени `root (UID 0)` внутри контейнера, на уровне хоста отображался как непривилегированный пользователь (например, `UID 100000`). Даже если он «вырвется» наружу, он не сможет смонтировать диск или изменить системные файлы.

`Применение Mandatory Access Control (AppArmor / SELinux)`: Эти подсистемы ядра позволяют создать профили, которые жестко запрещают монтирование блочных устройств или обращение к `/dev`, даже если у процесса есть нужная `capability cap_sys_admin`.

Часть 2.

Роль `seccomp` в защите ядра.

`seccomp (Secure Computing)` — это брандмауэр для системных вызовов (`syscalls`). Ядро Linux имеет более 300 системных вызовов, но обычному контейнеру для работы нужно около половины. Docker по умолчанию использует seccomp-профиль, который блокирует ~44 потенциально опасных системных вызова.

Одним из таких заблокированных вызовов является `unshare`, который используется для создания новых пространств имен (`namespaces`).

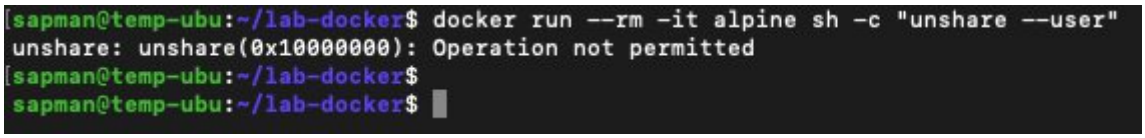
Поведение по умолчанию (Заблокировано)

Создание нового `User Namespace` в обычном контейнере:

```shell

      docker run --rm -it alpine sh -c "unshare --user"

```

Ошибка: «`unshare: unshare(0x10000000): Operation not permitted`»

Профиль `seccomp` заблокировал системный вызов.



Отключение `seccomp`

Запуск контейнер без профиля `seccomp`:

```shell

      docker run --rm -it --security-opt seccomp=unconfined alpine sh -c
"unshare --user && echo 'User namespace created successfully!'"

```

Выполнение команды не было заблокировано.

Создание вложенных пространств имен (`nested namespaces`, которые делает `unshare`) исторически является огромной поверхностью для атак на ядро Linux. Множество уязвимостей эскалации привилегий `(Local Privilege Escalation, LPE)` связано с тем, что злоумышленник создает собственное пространство имен, получает там права "виртуального" `root` и использует их для атаки на плохо изолированные подсистемы ядра.

Вывод:

Профиль `seccomp` в Docker играет роль превентивной защиты (`Defense in Depth`). Блокируя некритичные для обычных веб-сервисов системные вызовы (такие как `unshare, kcmp, ptrace`), Docker радикально снижает вероятность того, что злоумышленник сможет эксплуатировать 0-day уязвимости (уязвимости нулевого дня) в ядре Linux для выхода из контейнера.

Дополнительная информация

Результаты остальных заданий ВШЭ размещены по ссылке:

https://github.com/sapsolman/netology-hse